$ ./introduce --self

Perbandingan Arsitektur Web: <Layered> vs <Non-Layered>

Tim Penyusun:

// Abdillah Ridho Alriza
// Faiz Muhammad Ghazali
// Hafiz Rifai
// Irsyad Afif Musyaffa
// Muhammad Fadey Rafif

connected [●]
online [●]

// Bab I

Latar Belakang dan Tujuan

// message 01

3 Peran

Tanggung Jawab Aplikasi Web

- Menampilkan antarmuka pengguna
- Menjalankan aturan bisnis
- Serta mengelola penyimpanan data pada basis data.

// message 02

Tiga Fokus Eksperimen Kelompok

- **Aplikasi web Non-Layered :** Membangun CRUD pelanggan dalam satu berkas PHP.
- **Aplikasi web 3-Layered :** Memisahkan lapisan via REST API berformat JSON.
- **Analisis Objektif :** Membandingkan kedua pendekatan berdasarkan kode yang telah dibuat

// SPESIFIKASI TUGAS & BASIS DATA

Studi Kasus dan Skema Data

~/ Aplikasi Customer Management System wajib memenuhi 6 fitur (Tambah, Tampil, Edit, Hapus, Pencarian, Validasi) dengan struktur tabel pelanggan berikut:

Kolom Tabel	Tipe Data SQL	Aturan & Constraint	Keterkaitan Fitur
id	INT AUTO_INCREMENT	Primary Key	Identifikasi Edit & Hapus
nama	VARCHAR(100)	Wajib diisi (NOT NULL)	Target filter pencarian
telepon	VARCHAR(20)	Wajib diisi (8–15 digit)	Validasi input pelanggan
email	VARCHAR(100)	**Wajib diisi & UNIQUE**	Pencarian & cek duplikasi
alamat	TEXT	Wajib diisi (NOT NULL)	Data domisili pelanggan
created_at	TIMESTAMP	Default waktu saat ini	Pencatatan waktu otomatis

// BAB II — LANDASAN TEORI

Konsep Arsitektur Tiga Lapisan

01

Presentation Layer

Antarmuka visual interaksi pengguna langsung.

HTML, CSS, JavaScript

02

Business Logic Layer

Pusat validasi aturan proses bisnis.

PHP Native Service

03

Data Access Layer

Pengelolaan penuh eksekusi kueri basisdata.

MySQL, PDO Repository

// BAB III — IMPLEMENTASI APLIKASI PERTAMA

Arsitektur Aplikasi Non-Layered

01

Client / Browser

Mengirim HTTP POST/GET langsung ke berkas tunggal.

02

MONOLITIK
190 BARIS

Server PHP (index.php)

Menyatukan koneksi mysqli, validasi, SQL, dan render HTML.

03

Database MySQL

Menyimpan tabel pelanggan pada db_pelanggan.sql.

Karakteristik Teknis Implementasi Non-Layered

Menggunakan pola Post/Redirect/Get serta menangani duplikasi email via tangkapan kode error MySQL 1062.

`1-Layer/index.php`

// BAB III — IMPLEMENTASI APLIKASI KEDUA

Arsitektur Aplikasi 3-Layer

Aplikasi dipisahkan ke dalam direktori frontend/ dan backend/ yang saling berkomunikasi melalui Fetch API dengan format JSON:

01

Presentation

`index.html & app.js`

Menangkap input pengguna lalu memanggil Fetch API.

02

API Controller

`backend/api.php`

Memetakan metode HTTP menuju service terkait.

03

Business Logic

`PelangganService.php`

Memvalidasi kelengkapan serta keunikan email.

04

Data Layer

`PelangganRepository.php`

Mengeksekusi kueri CRUD melalui koneksi PDO.

// KONTRAK KOMUNIKASI CLIENT-SERVER

Desain REST API dan Alur Kerja

Spesifikasi Endpoint (backend/api.php)

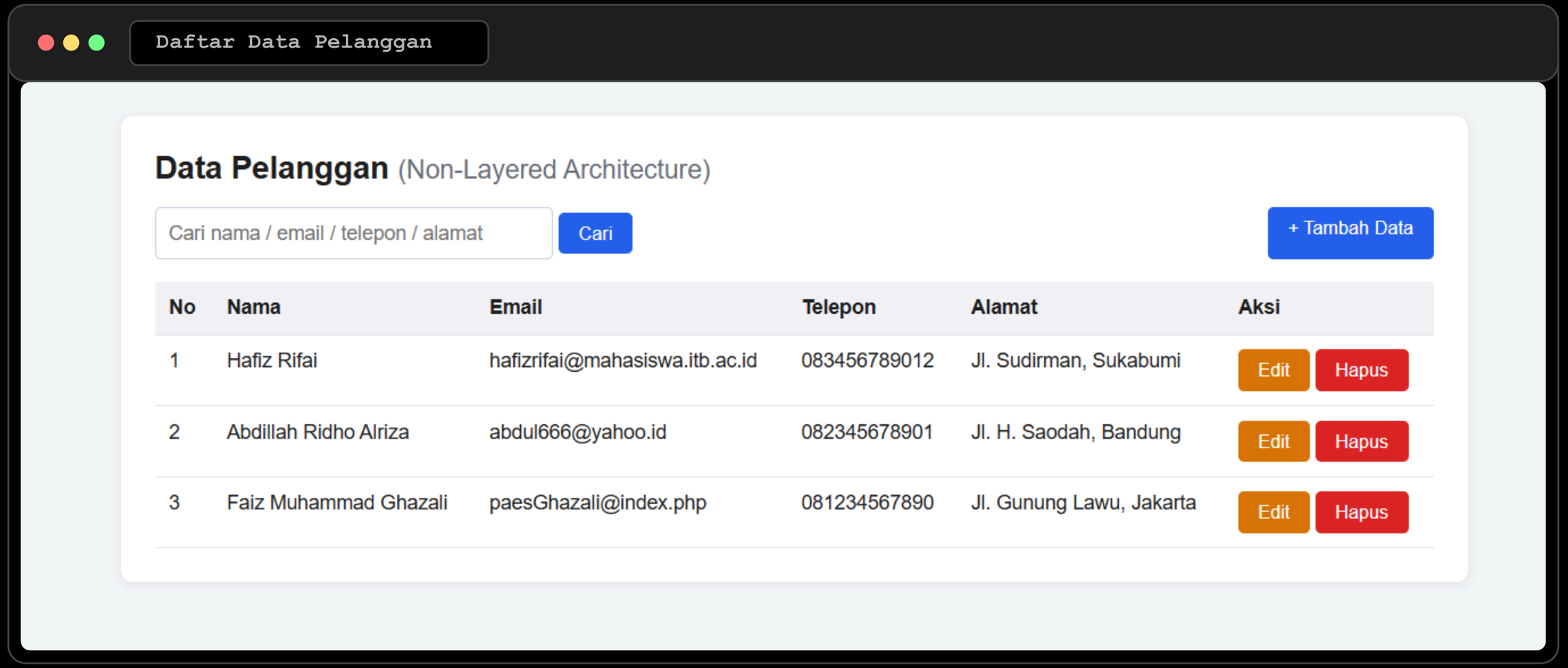
Metode	Parameter	Fungsi Utama	Status HTTP
GET	?search=...	Tampil & cari data	200
POST	Body JSON	Tambah pelanggan	201 / 422
PUT	?id=...	Ubah pelanggan	200 / 404 / 422
DELETE	?id=...	Hapus pelanggan	200 / 404

Alur Tambah Pelanggan (POST)

1. **Request JSON:** Skrip app.js mengirim payload ke endpoint API.
2. **Routing API:** Berkas api.php memanggil fungsi createCustomer.
3. **Validasi Bisnis:** Service memeriksa kelengkapan dan keunikan email.
4. **Query PDO:** Repository mengeksekusi prepared statement INSERT.
5. **Respons Klien:** API mengembalikan kode 201 lalu tabel dimuat.

// BAB IV — HASIL IMPLEMENTASI (NON-LAYERED)

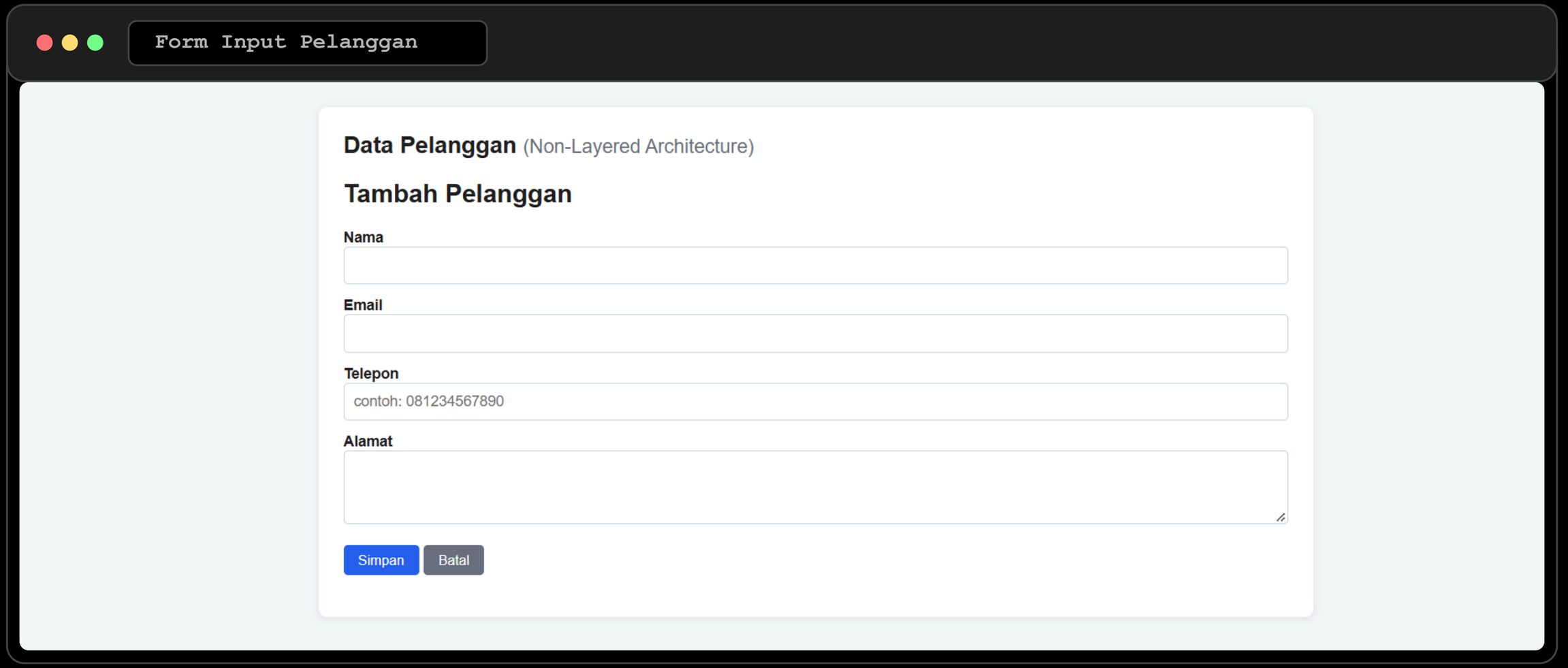
Tampilan Aplikasi Non-Layered #1

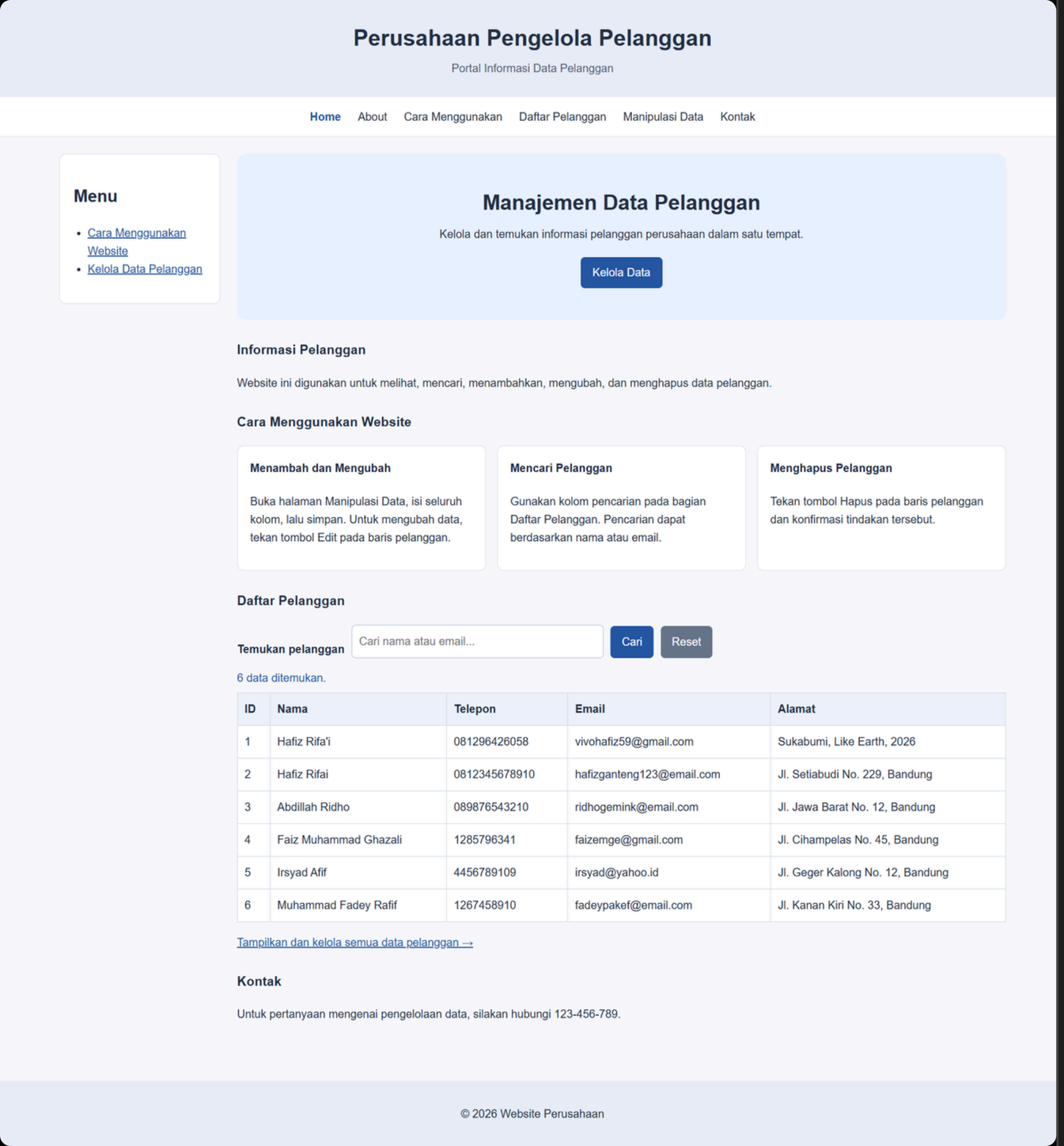


// BAB IV — HASIL IMPLEMENTASI (NON-LAYERED)

Tampilan Aplikasi Non-Layered #2

Form Input Pelanggan



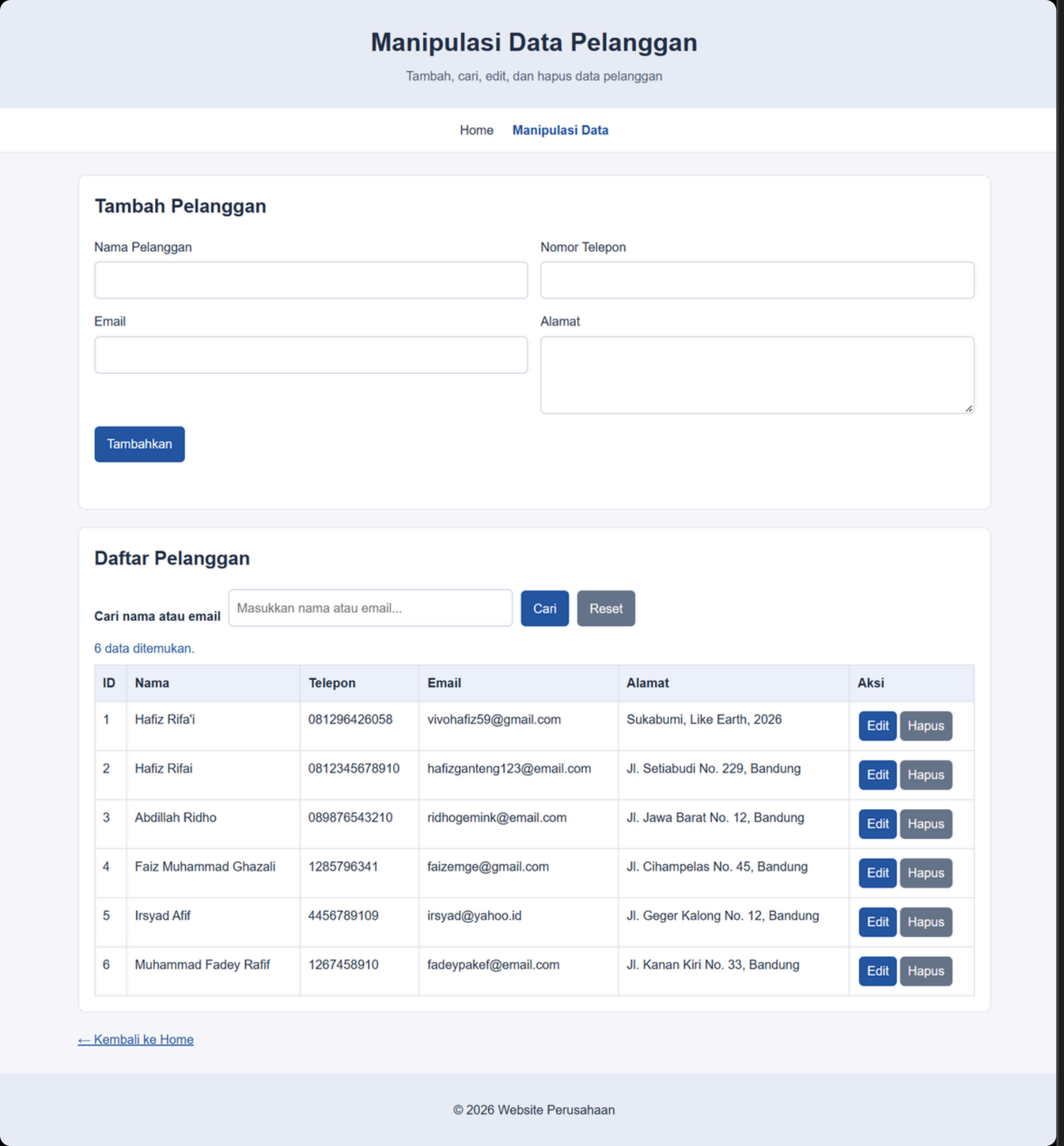


Halaman Portal Utama

// BAB IV — HASIL IMPLEMENTASI (3-LAYER ARCHITECTURE)

Tampilan Aplikasi 3-Layer #1

Halaman Portal Utama

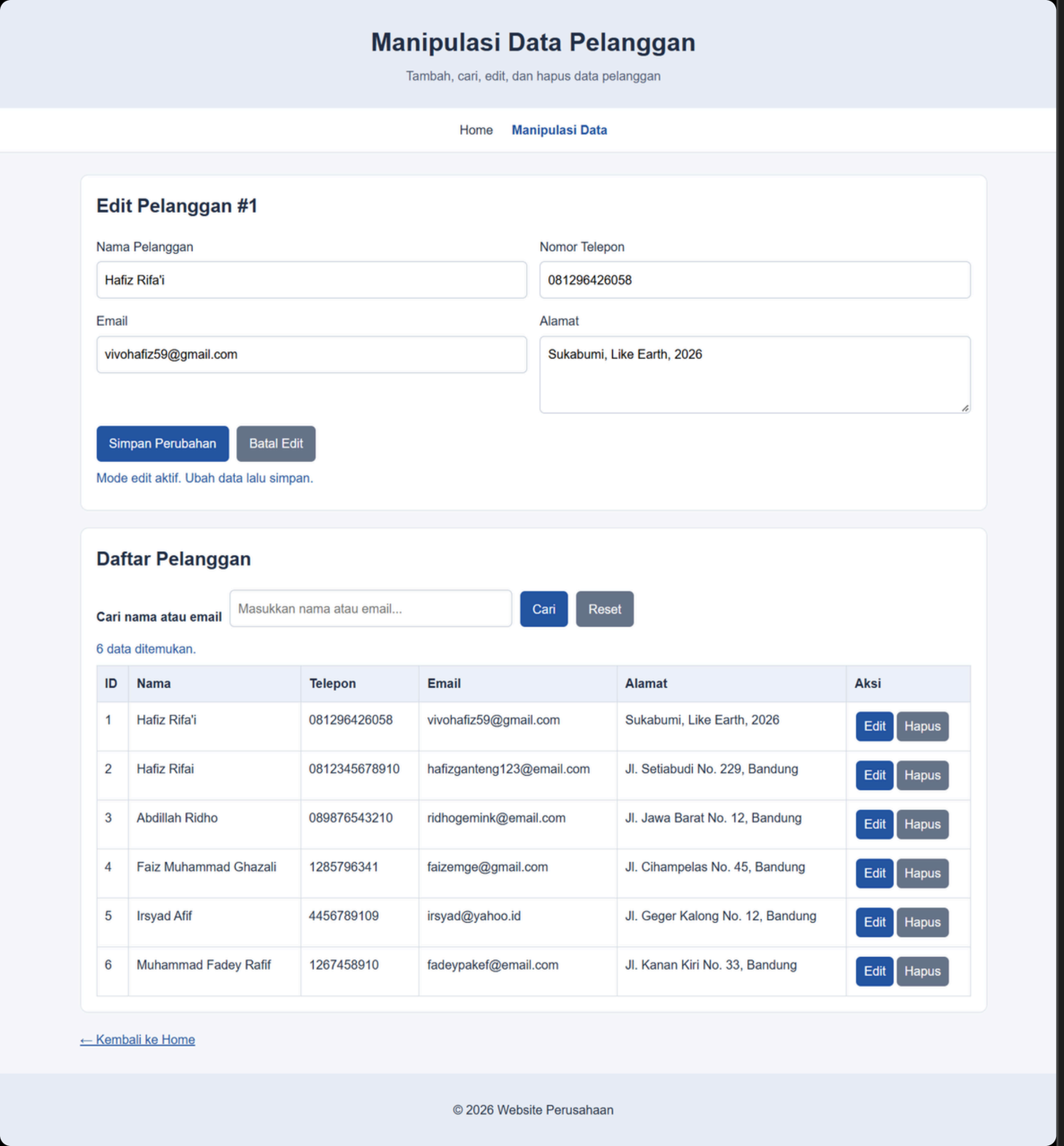


Halaman Manipulasi Data

// BAB IV — HASIL IMPLEMENTASI
(3-LAYER ARCHITECTURE)

Tampilan Aplikasi 3-Layer #2

Halaman Manipulasi Data (**Tambah**)



Halaman Manipulasi Data

// BAB IV — HASIL IMPLEMENTASI
(3-LAYER ARCHITECTURE)

Tampilan Aplikasi 3-Layer #3

Halaman Manipulasi Data (**Edit**)

// BAB V — ANALISIS PENGUJIAN & METRIK KODE

Perbandingan Kuantitatif Kode

01 Non-Layered Architecture (1 Berkas PHP Tunggal: index.php)

190 Baris Kode (HTML + Logika + SQL Menyatu)



02 3-Layer Backend (4 Berkas PHP: API 54, Service 68, Repository 74, Database 29)

225 Baris Kode (Keluaran Murni JSON)



03 3-Layer Frontend (4 Berkas Antarmuka: 2 HTML + 1 JS + 1 CSS)

±300 Baris Kode (Terpisah dari PHP)



// BAB V — EVALUASI ARSITEKTURAL

Perbandingan Kualitatif Arsitektur

Aspek	Non-Layered Architecture	3-Layer Architecture
Pembuatan Awal	Sangat cepat dibuat tanpa kontrak API.	Lebih lama merancang lapisan modul.
Keterbacaan Kode	Memburuk saat fitur aplikasi bertambah besar.	Terstruktur rapi dengan satu tanggung jawab.
Isolasi Bug	Sulit dilokalisasi karena kode saling bercampur.	Mudah dilacak langsung pada lapisan terkait.
Pengujian Logika	Sulit diuji tanpa melalui antarmuka HTML.	Dapat diuji independen menggunakan Postman
Kerja Tim	Rawan konflik karena mengedit berkas sama.	Mudah dibagi per peran lapisan.

BAB VI — KESIMPULAN

Empat Poin Kesimpulan Utama

01 — PEMENUHAN FUNGSIONALITAS

Enam Fitur Wajib Berjalan Penuh

Kedua arsitektur berhasil memenuhi fitur CRUD pencarian validasi.

02 — KOMPROMI KECEPATAN VS STRUKTUR

Risiko Spaghetti Code Monolitik

Non-Layered cepat dibuat awal namun menyulitkan pemeliharaan kode.

03 — KELEBIHAN 3-LAYER ARCHITECTURE

Modularitas dan Reusability API

Memudahkan isolasi bug pengujian Postman pembagian tugas tim.

04 — KEAMANAN & SKALABILITAS

Isolasi Penuh Lapisan Database

Data Layer terlindungi dari akses langsung sisi browser.

// end_of_deck

Thank You

[questions ?]

exit code: 0 · runtime: ~18 min